



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente
Ingeniería en Desarrollo de Software

Asignatura: Programación orientada a objetos

Unidad #4

Fecha: Del 07/09/26 hasta 13/09/26

Tema: **POO Avanzada: Herencia, Polimorfismo y Genéricos**

Autor:

Introducción:

En este documento exploraremos cuatro pilares fundamentales de la programación orientada a objetos en Java: el polimorfismo, las interfaces, las clases abstractas y el encapsulamiento. Estos conceptos constituyen la base para diseñar sistemas más flexibles, reutilizables y mantenibles, permitiendo que los programas puedan adaptarse a diferentes escenarios sin necesidad de reescribir grandes porciones de código.

El polimorfismo nos enseña cómo un mismo método puede comportarse de diversas formas según el objeto que lo invoque. Las interfaces nos muestran cómo definir contratos de comportamiento que diferentes clases pueden implementar, logrando polimorfismo incluso entre clases no relacionadas. Por su parte, las clases abstractas permiten establecer plantillas que combinan comportamientos generales y métodos obligatorios para las subclases, asegurando una jerarquía coherente en el diseño del software. Finalmente, el encapsulamiento nos enseña a proteger el estado interno de los objetos, controlando el acceso a sus atributos y exponiendo únicamente lo necesario a través de métodos, lo cual incluye el uso de clases internas, anidadas y genéricos para organizar y reutilizar el código de forma más segura.

Este material ha sido diseñado para ofrecerte una comprensión sólida de estos temas y ayudarte a incorporarlos en tus propios proyectos de programación. Te animamos a revisar con atención los ejemplos, diagramas y ejercicios propuestos, ya que son esenciales para afianzar tu aprendizaje y desarrollar las competencias necesarias en tu trayectoria como futuro profesional del desarrollo de software.



4.4. Polimorfismo

El **polimorfismo** es la capacidad que tienen los objetos en la Programación Orientada a Objetos (POO) de **adoptar múltiples formas**. En otras palabras: una misma referencia (variable) puede apuntar a objetos de diferentes clases, siempre que esas clases estén relacionadas por **herencia** o por **interfaces**.

Esto permite que un mismo método pueda comportarse de distintas maneras dependiendo del tipo real del objeto que lo invoque.

Idea central

- **Herencia:** una subclase puede sobrescribir métodos de la superclase.
- **Polimorfismo:** una misma llamada al método ejecuta la versión correspondiente al **tipo real del objeto**, no al tipo de la referencia.

Ejemplo de la vida real:

- Un **Animal** puede hacer un sonido.
- Un **Perro** ladra, un **Gato** maúlla.
- Cuando se invoca `hacerSonido()`, el comportamiento depende del objeto concreto, aunque lo llamemos desde una variable de tipo `Animal`.

```
1 class Animal {
2 >     public void hacerSonido() { System.out.println("El animal hace un sonido"); }
5 }
6
7 class Perro extends Animal {
8     @Override
9 >     public void hacerSonido() { System.out.println("El perro ladra: ¡Guau!"); }
12 }
13
14 class Gato extends Animal {
15     @Override
16 >     public void hacerSonido() { System.out.println("El gato maúlla: ¡Miau!"); }
19 }
20
21 > public class EjemploPolimorfismo {
22 >     public static void main(String[] args) {
23         Animal a1 = new Perro(); // Variable de tipo Animal, objeto Perro
24         Animal a2 = new Gato();  // Variable de tipo Animal, objeto Gato
25
26         a1.hacerSonido(); // "El perro ladra: ¡Guau!"
27         a2.hacerSonido(); // "El gato maúlla: ¡Miau!"
28     }
29 }
30
```

Aunque a1 y a2 son de tipo Animal, el método ejecutado corresponde al **tipo real del objeto** (Perro o Gato). Eso es polimorfismo

Ventajas del polimorfismo

1. **Reutilización de código:** podemos trabajar con la superclase sin preocuparnos por las subclases específicas.
2. **Flexibilidad y extensibilidad:** agregar nuevas subclases no obliga a modificar el código que ya usa polimorfismo.
3. **Mantenimiento más fácil:** se evita la sobrecarga de condicionales (if, switch) para decidir el comportamiento.

Resumen para recordar

- Una **referencia** de la superclase puede almacenar objetos de cualquier subclase.
- El método ejecutado siempre depende del **objeto real** (dinámicamente).
- Polimorfismo = “un mismo mensaje, diferentes respuestas”.

4.4.1. Asignación de objetos a variables de su superclase.

En Java (y en la mayoría de lenguajes orientados a objetos), una variable de tipo superclase puede almacenar (referenciar) un objeto de cualquiera de sus subclases.

Esto se debe a la relación de herencia:

- Una subclase es un tipo especial de su superclase.



- Por lo tanto, una variable de la superclase puede apuntar a cualquier instancia de sus subclases.

A esto se le llama upcasting (conversión hacia arriba en la jerarquía de clases).

```
1 class Animal { 3 usages 2 inheritors 2 franciscomorales25 *
2     public void hacerSonido() { 2 usages 2 overrides new *
3         System.out.println("El animal hace un sonido");
4     }
5 }
6
7 class Perro extends Animal { 1 usage 2 franciscomorales25 *
8     @Override 2 usages new *
9     public void hacerSonido() {
10         System.out.println("El perro ladra: ¡Guau!");
11     }
12 }
13
14 class Gato extends Animal { 1 usage new *
15     @Override 2 usages new *
16     public void hacerSonido() {
17         System.out.println("El gato maúlla: ¡Miau!");
18     }
19 }
20
21 public class AsignacionSuperclase { 2 franciscomorales25 *
22     public static void main(String[] args) { new *
23         Animal miAnimal; // Variable de tipo Animal (superclase)
24
25         miAnimal = new Perro(); // Puede almacenar un Perro
26         miAnimal.hacerSonido(); // Guau!
27
28         miAnimal = new Gato(); // Ahora apunta a un Gato
29         miAnimal.hacerSonido(); // Miau!
30     }
```

Puntos importantes

1. **La variable es de tipo superclase (Animal)**, pero puede contener objetos de Perro, Gato u otras subclases.



2. **El método ejecutado corresponde al objeto real**, no al tipo de la variable.
3. Esto permite programar de forma más **genérica y flexible**, trabajando con el tipo padre en lugar de con cada subclase por separado.

Recordatorio

- **Superclase**: clase base o padre.
- **Subclase**: clase hija que hereda atributos y métodos.
- **Asignación**: cuando declaramos una variable de tipo **superclase** y le asignamos un objeto de tipo **subclase**.
- En Java esto es **upcasting automático**, porque toda subclase es también un tipo de su superclase.

```
Animal miAnimal = new Perro(); // Upcasting
```

Aquí sucede lo siguiente:

- La variable miAnimal es de tipo **Animal**.
- El objeto creado es un **Perro**.
- La referencia trata al Perro como un Animal, pero al ejecutar métodos sobrescritos se comportará como Perro.

Ventajas del uso de variables de superclase

1. **Generalización del código**
No es necesario declarar variables específicas para cada subclase. Ejemplo: en vez de tener listas separadas de Perro y Gato, podemos tener una lista de Animal.
2. **Uso de colecciones polimórficas**



```
List<Animal> animales = new ArrayList<>();
animales.add(new Perro());
animales.add(new Gato());
animales.add(new Perro());
```

Luego recorremos la lista y cada objeto se comporta según su tipo real.

3 .Flexibilidad para extender programas

Si más adelante agregamos una nueva subclase (ejemplo: Vaca), el código que usa Animal no cambia.

Diferencia entre tipo de referencia y tipo de objeto

- Tipo de referencia: el tipo declarado de la variable (ejemplo: Animal).
- Tipo de objeto: la clase del objeto real creado en memoria (ejemplo: Perro).

El compilador usa el tipo de referencia para validar qué métodos pueden llamarse. La JVM (tiempo de ejecución) decide qué versión del método se ejecuta según el tipo real del objeto.

```
Animal miAnimal = new Perro();
miAnimal.hacerSonido(); // Ejecuta la versión de Perro

miAnimal = new Gato();
miAnimal.hacerSonido(); // Ejecuta la versión de Gato
```

Limitación importante

Cuando usamos una **variable de superclase**, solo podemos acceder a los **métodos y atributos definidos en la superclase** (aunque el objeto sea de una subclase).




```

class Perro extends Animal {
    public void moverCola() {
        System.out.println("El perro mueve la cola");
    }
}

Animal miAnimal = new Perro();
miAnimal.hacerSonido();    // OK
miAnimal.moverCola();     // ERROR: no está definido en Animal

```

Aunque el objeto real sea un Perro, la variable de tipo Animal no permite acceder directamente a métodos exclusivos de Perro.

Downcasting (relacionado)

Si necesitamos acceder a métodos propios de la subclase, debemos hacer un downcasting (conversión hacia abajo):

```

Animal miAnimal = new Perro();
((Perro) miAnimal).moverCola(); // Conversión explícita a Perro

```

El downcasting puede provocar errores en tiempo de ejecución si la conversión no es válida.

Analogía de la vida real

Imagina que tienes un **cajón para “Animales”**:

- Dentro puedes guardar un Perro o un Gato , porque ambos son Animales.
- Desde afuera, solo puedes decir “esto es un Animal y puede hacerSonido()”.
- Si quieres algo específico (ejemplo: que el Perro mueva la cola), necesitas abrir el cajón y comprobar qué animal guardaste realmente (**downcasting**).



Resumen extendido

- Asignar una subclase a una variable de superclase = **upcasting**.
- Es **automático y seguro**.
- El **método ejecutado** depende del objeto real, no de la referencia.
- La referencia de la superclase solo permite usar lo definido en la superclase.
- Para acceder a lo exclusivo de la subclase se requiere **downcasting**.

4.5. Interfaces

Qué es una interfaz?

Una **interfaz** en Java es un **contrato** que establece un conjunto de métodos que una clase debe implementar.

- Define **qué debe hacer** una clase, pero no dice **cómo debe hacerlo**.
- Una interfaz asegura que todas las clases que la implementen tengan los mismos métodos disponibles.

Ejemplo de la vida real:
Piensa en una **licencia de conducir**: obliga a que la persona sepa manejar, pero no especifica cómo conduce cada quien.

Características principales

- Se declaran con la palabra clave **interface**.
- Los métodos son **public abstract** por defecto (aunque no se escriba).
- Los atributos son siempre **public static final** (constantes).
- Una interfaz no puede **instanciarse** directamente.
- Una clase puede **implementar múltiples interfaces** (supera la limitación de la herencia simple).

Reglas fundamentales de las interfaces



1. Se definen con interface.
2. Se implementan en clases usando la palabra implements.
3. Una clase puede implementar más de una interfaz.
4. Los métodos definidos en la interfaz deben sobrescribirse en la clase.
5. No se pueden crear objetos de una interfaz.

4.5.1. Definición e implementación de interfaz

Definición de una interfaz

java

```
public interface Volador {  
    void volar(); // método abstracto (sin cuerpo)  
}
```

Aquí solo se declara qué debe hacer (volar()), pero no cómo.

Implementación en clases

```
class Pajaro implements Volador {  
    @Override  
    public void volar() {  
        System.out.println("El pájaro vuela con sus alas");  
    }  
}  
  
class Avion implements Volador {  
    @Override  
    public void volar() {  
        System.out.println("El avión vuela con sus motores");  
    }  
}
```



- Ambas clases cumplen el contrato de Volador.
- Cada una implementa el método de manera distinta.

```
public class Demo {  
    public static void main(String[] args) {  
        Pajaro p = new Pajaro();  
        Avion a = new Avion();  
  
        p.volar(); // "El pájaro vuela con sus alas"  
        a.volar(); // "El avión vuela con sus motores"  
    }  
}
```

Aquí trabajamos con objetos concretos, cada uno implementa la interfaz a su manera.

4.5.2. Interfaces y polimorfismo

El **polimorfismo con interfaces** consiste en que:

- Una variable de tipo **interfaz** puede referenciar a cualquier objeto de una clase que la implemente.
- El método que se ejecuta depende del **objeto real**, no del tipo de la referencia.

Recordatorio: ¿qué es polimorfismo?

El **polimorfismo** significa “*muchas formas*”. En programación orientada a objetos:

- Un mismo método puede tener diferentes comportamientos según el objeto que lo invoque.



- En el caso de las **interfaces**, el polimorfismo permite que clases totalmente distintas se usen de manera uniforme siempre que implementen la misma interfaz.

Concepto central

- Una referencia de tipo interfaz puede almacenar un objeto de cualquier clase que la implemente.
- Al invocar un método desde esa referencia, la versión ejecutada será la del objeto real.
- Esto facilita escribir programas genéricos y extensibles, sin depender de clases concretas.



java

```
interface Volador {  
    void volar();  
}  
  
class Pajaro implements Volador {  
    @Override  
    public void volar() {  
        System.out.println("El pájaro vuela con sus alas");  
    }  
}  
  
class Avion implements Volador {  
    @Override  
    public void volar() {  
        System.out.println("El avión vuela con sus motores");  
    }  
}  
  
public class DemoPolimorfismo {  
    public static void main(String[] args) {  
        Volador v1 = new Pajaro(); // referencia de interfaz  
        Volador v2 = new Avion();  // referencia de interfaz  
  
        v1.volar(); // "El pájaro vuela con sus alas"  
        v2.volar(); // "El avión vuela con sus motores"  
    }  
}
```

Aquí el contrato (volar) es único, pero la acción depende del objeto.



Uso con colecciones

Una de las aplicaciones más prácticas es el manejo de listas de objetos heterogéneos que comparten una interfaz.

```
List<Volador> voladores = new ArrayList<>();
voladores.add(new Pajaro());
voladores.add(new Avion());

for (Volador v : voladores) {
    v.volar(); // cada objeto se comporta según su tipo real
}
```

El programa no necesita saber si el objeto es un Pajaro o un Avion, simplemente lo trata como Volador.

Ventajas del polimorfismo con interfaces

1. Desacoplamiento: el código no depende de clases concretas, sino de la interfaz.
2. Extensibilidad: nuevas clases pueden añadirse sin modificar el código existente.
3. Flexibilidad: distintas clases no relacionadas jerárquicamente pueden trabajar juntas.
4. Reutilización: un mismo método puede recibir objetos de diferentes tipos que cumplan la misma interfaz.

Ejemplo con múltiples interfaces

El polimorfismo funciona incluso cuando una clase implementa varias interfaces:



```

interface Volador { void volar(); }
interface Nadador { void nadar(); }

class Pato implements Volador, Nadador {
    @Override
    public void volar() {
        System.out.println("El pato vuela bajo");
    }
    @Override
    public void nadar() {
        System.out.println("El pato nada en el lago");
    }
}

```

```

java

public class Demo {
    public static void main(String[] args) {
        Volador v = new Pato();
        Nadador n = new Pato();

        v.volar(); // "El pato vuela bajo"
        n.nadar(); // "El pato nada en el lago"
    }
}

```

El mismo objeto (Pato) se comporta como Volador o como Nadador, dependiendo de la referencia.



Comparación con polimorfismo por herencia

- Con herencia, el polimorfismo funciona entre superclase y subclase.
- Con interfaces, funciona entre clases que no tienen ninguna relación jerárquica, pero que comparten un contrato.

Ejemplo:

- Un Avion y un Pajaro no tienen relación de herencia, pero ambos cumplen el contrato Volador.

Analogía de la vida real

Imagina un torneo deportivo donde todos los participantes deben firmar un contrato para competir en la categoría “correr”.

- No importa si es un humano, un perro o un robot.
- Mientras todos implementen la acción correr(), el juez (el programa) los trata de la misma forma.

Resumen

- El polimorfismo con interfaces permite trabajar con diferentes clases bajo un mismo contrato.
- Se logra porque una referencia de interfaz puede apuntar a cualquier objeto que la implemente.
- Facilita el diseño de programas flexibles, extensibles y desacoplados.
- Es la base de muchas librerías y frameworks en Java (ejemplo: List, Comparable, Runnable).

4.6. Clases y métodos abstractos

Definición de clase abstracta

Una **clase abstracta** en Java es una clase que sirve como **plantilla** para otras clases.

- Puede contener métodos **abstractos** (sin cuerpo) y métodos **concretos** (con implementación).
- **No puede instanciarse directamente**, es decir, no se pueden crear objetos de ella.



- Su objetivo es establecer una **estructura común** para las subclases.

Ejemplo en la vida real:
Imagina la clase abstracta “**Empleado**”: define que todos los empleados tienen un nombre y un salario, pero cada tipo de empleado (Gerente, Programador) implementa su forma de trabajar.

Definición de método abstracto

Un **método abstracto** es un método declarado en una clase abstracta sin cuerpo (sin { }).

- Solo indica la **firma** (nombre, parámetros y tipo de retorno).
- Obliga a las subclases a **sobrescribirlo**.

Ejemplo:

Todas las clases hijas (Circulo, Rectangulo, etc.) deben implementar calcularArea()

Características de las clases abstractas

1. Se declaran con la palabra clave **abstract**.
2. Pueden tener **constructores, atributos y métodos implementados**.
3. Una subclase debe sobrescribir todos los métodos abstractos, o declararse abstracta también.
4. Permiten definir **comportamiento común** y dejar detalles específicos a las subclases.
5. No se pueden instanciar directamente, pero sí a través de sus subclases



Ejemplo completo

```
// Clase abstracta
abstract class Animal {
    String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    // Método abstracto
    public abstract void hacerSonido();

    // Método concreto
    public void dormir() {
        System.out.println(nombre + " está durmiendo");
    }
}

// Subclases que implementan el método abstracto
class Perro extends Animal {
    public Perro(String nombre) { super(nombre); }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " ladra: ¡Guau!");
    }
}

class Gato extends Animal {
    public Gato(String nombre) { super(nombre); }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " maúlla: ¡Miau!");
    }
}
```




```

public class DemoAbstracta {
    public static void main(String[] args) {
        // Animal a = new Animal("Genérico"); // ERROR: no se puede instanciar
        Animal perro = new Perro("Firulais");
        Animal gato = new Gato("Michi");

        perro.hacerSonido(); // Firulais ladra: ¡Guau!
        gato.hacerSonido();  // Michi maúlla: ¡Miau!
        perro.dormir();      // Firulais está durmiendo
    }
}

```

Diferencias entre clases abstractas e interfaces

Aspecto	Clase abstracta	Interfaz
Instanciación	X No	X No
Herencia múltiple	X Solo una clase abstracta	✓ Varias interfaces
Métodos	✓ Abstractos y concretos	X Solo abstractos (salvo default y static)
Atributos	✓ Variables normales y constantes	X Solo constantes (final)
Uso	Plantilla base para clases relacionadas	Contrato común para clases distintas

Clases abstractas = molde incompleto que define atributos + lógica parcial.
 Interfaces = contrato puro de comportamiento.

Analogía

Una **clase abstracta** es como un **boceto arquitectónico**:

- Define la estructura principal (habitaciones, baños, puertas).
- Cada constructor (subclase) decide los detalles (colores, materiales, decoración).



Ventajas de las clases abstractas

1. Permiten **reutilizar código** común en las subclases.
2. Garantizan que las subclases implementen ciertos métodos obligatorios.
3. Son útiles para construir **jerarquías de herencia** coherentes.
4. Combinan lo mejor de dos mundos: métodos abstractos (flexibilidad) + métodos concretos (reutilización).

Buenas prácticas

- Usar clases abstractas cuando las clases hijas comparten **estado y comportamiento común**.
- Usar interfaces cuando solo se quiere definir un **contrato de comportamiento**.

Combinar ambas:

- Clase abstracta Animal con atributos y métodos base.
- Interfaz Volador implementada solo por los animales que pueden volar.

4.7. Encapsulamiento

¿Qué es el encapsulamiento?

El encapsulamiento es el principio de la Programación Orientada a Objetos (POO) que consiste en ocultar el estado interno de un objeto (sus atributos) y exigir que toda interacción con ese estado se realice a través de métodos definidos por la propia clase.

En otras palabras: los datos de un objeto no se acceden ni se modifican directamente desde fuera de la clase, sino mediante métodos públicos (generalmente llamados getters y setters) que controlan cómo se leen o cambian esos datos.



Ejemplo de la vida real:

Piensa en un cajero automático: no puedes meter la mano directamente en la bóveda del banco para tomar o depositar dinero. Debes usar la interfaz del cajero (el teclado y la pantalla), que valida cada operación antes de modificar tu saldo.

Idea central

- **Ocultamiento de datos:** los atributos de una clase se declaran como `private` y no pueden accederse directamente desde fuera de ella.
- **Acceso controlado:** los datos se exponen mediante métodos públicos que validan la información antes de leerla o modificarla.
- **Protección de la integridad:** cada clase es responsable de mantener su propio estado interno consistente.

Características principales

- Los atributos se declaran como `private`.
- Se exponen métodos públicos `getX()` y `setX()` para leer y modificar los atributos.
- Los métodos `setX()` pueden incluir validaciones antes de modificar el valor.
- El resto del programa nunca accede directamente a los atributos, solo a través de los métodos.

Ejemplo de una clase encapsulada:




```
java
public class CuentaBancaria {

    private double saldo; // atributo privado

    public double getSaldo() {
        return saldo;
    }

    public void depositar(double monto) {
        if (monto > 0) {
            saldo += monto;
        }
    }
}
```

El atributo saldo es privado; solo puede leerse con getSaldo() y modificarse de forma controlada con depositar(), que valida que el monto sea positivo antes de sumarlo.

Setters con validación

Una de las mayores ventajas del encapsulamiento es que un setter puede rechazar valores inválidos antes de que lleguen a modificar el estado del objeto:



```

java
public class CuentaBancaria {
    private double saldo;

    public void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        } else {
            System.out.println("Saldo invalido");
        }
    }
}

```

Si se intenta asignar un saldo negativo, el método lo rechaza y el objeto nunca queda en un estado inconsistente. Esto sería imposible de garantizar si saldo fuera un atributo público modificable libremente desde cualquier parte del programa.

Ventajas del encapsulamiento

1. **Protección del estado interno:** nadie puede modificar el estado de un objeto sin pasar por las validaciones definidas en la clase.
2. **Mantenibilidad:** la clase puede cambiar cómo almacena internamente sus datos sin afectar el código que la usa, siempre que los métodos públicos se mantengan iguales.
3. **Bajo acoplamiento:** se evita que el resto del programa dependa de la forma exacta en que se implementan los atributos.
4. **Facilidad de depuración:** los errores relacionados con datos inválidos se detectan en un solo lugar: el setter correspondiente.

Analogía de la vida real

Imagina el tablero de control de un avión: los pilotos interactúan con botones, palancas e indicadores (la interfaz pública), pero nunca manipulan directamente los cables, sensores o motores (el estado interno). Esa separación evita errores



catastróficos y permite a los ingenieros mejorar los sistemas internos sin cambiar la forma en que el piloto opera el avión.

Resumen para recordar

- Encapsulamiento = ocultar datos + exponer comportamiento controlado.
- Atributos privados (private), acceso mediante métodos públicos (getters/setters).
- Los setters pueden validar datos antes de modificar el estado del objeto.
- Reduce el acoplamiento y facilita el mantenimiento del código.

4.7.1. Uso de clases internas y anidadas

Java permite declarar una clase dentro de otra clase. Según cómo se declare, se distinguen dos tipos principales: clases internas (no estáticas) y clases anidadas estáticas.

Clases internas (inner classes)

Una clase interna se declara dentro de otra clase, sin la palabra clave static. Cada instancia de la clase interna está ligada a una instancia concreta de la clase externa, por lo que puede acceder directamente a sus atributos y métodos, incluso a los privados.

```
java
public class Motor {

    private String tipo;

    class Piston {
        void mover() {
            System.out.println("Moviendo piston del motor " + tipo);
        }
    }
}
```

Para crear un objeto de la clase interna se necesita primero un objeto de la clase externa:




```
java
```

```
Motor motor = new Motor();  
Motor.Piston piston = motor.new Piston();  
piston.mover();
```

Clases anidadas estáticas (static nested classes)

Una clase anidada estática se declara con la palabra clave static dentro de otra clase. A diferencia de la clase interna, no está ligada a ninguna instancia de la clase externa y no puede acceder directamente a sus atributos de instancia.

```
java
```

```
public class Caja {  
  
    static class Contenido {  
        String descripcion;  
  
        Contenido(String descripcion) {  
            this.descripcion = descripcion;  
        }  
    }  
}
```

Se instancia sin necesidad de un objeto de la clase externa:

```
java
```

```
Caja.Contenido c = new Caja.Contenido("Libros");  
System.out.println(c.descripcion);
```



Diferencias entre clase interna y clase anidada estática

- **Clase interna (no estática):** necesita una instancia de la clase externa para existir; puede acceder a los atributos de esa instancia.
- **Clase anidada estática:** no necesita una instancia de la clase externa; solo puede usar los miembros static de esta.

¿Para qué sirven?

- Agrupar clases que solo tienen sentido dentro del contexto de otra clase.
- Mejorar la organización y el encapsulamiento del código.
- Ocultar detalles de implementación que no deben usarse fuera de la clase externa.
- Evitar contaminar el espacio de nombres del paquete con clases auxiliares.

Analogía de la vida real

Piensa en un motor (clase externa) y su pistón (clase interna): un pistón no existe por sí solo, siempre pertenece a un motor específico y depende de sus características. En cambio, una pieza de repuesto genérica (clase anidada estática) puede fabricarse y guardarse en el catálogo del fabricante sin depender de un motor en particular.

Resumen para recordar

- Clase interna: ligada a una instancia de la clase externa; accede a sus atributos.
- Clase anidada estática: independiente de instancias; se comporta como una clase normal dentro de un espacio de nombres.
- Ambas ayudan a organizar mejor el código y a reforzar el encapsulamiento.

4.7.2. Genéricos

¿Qué son los genéricos?

Los genéricos permiten que una clase, interfaz o método trabajen con distintos tipos de datos sin perder la seguridad de tipos en tiempo de compilación. En lugar de



escribir una clase distinta para cada tipo de dato, se escribe una sola clase parametrizada con un tipo genérico (por convención T, E, K, V).

Ejemplo de la vida real:

Piensa en una caja de mudanza: la misma caja física puede usarse para guardar libros, ropa o utensilios de cocina. La caja no cambia, solo cambia lo que guardamos dentro. Los genéricos funcionan igual: la misma clase Caja puede almacenar un String, un Integer o cualquier otro tipo, definido al momento de usarla.

Clase genérica

```
java
public class Caja<T> {

    private T contenido;

    public void guardar(T contenido) {
        this.contenido = contenido;
    }

    public T obtener() {
        return contenido;
    }
}
```

El tipo T es un parámetro que se define al crear el objeto. Java verifica en tiempo de compilación que solo se guarde y se obtenga el tipo indicado.



java

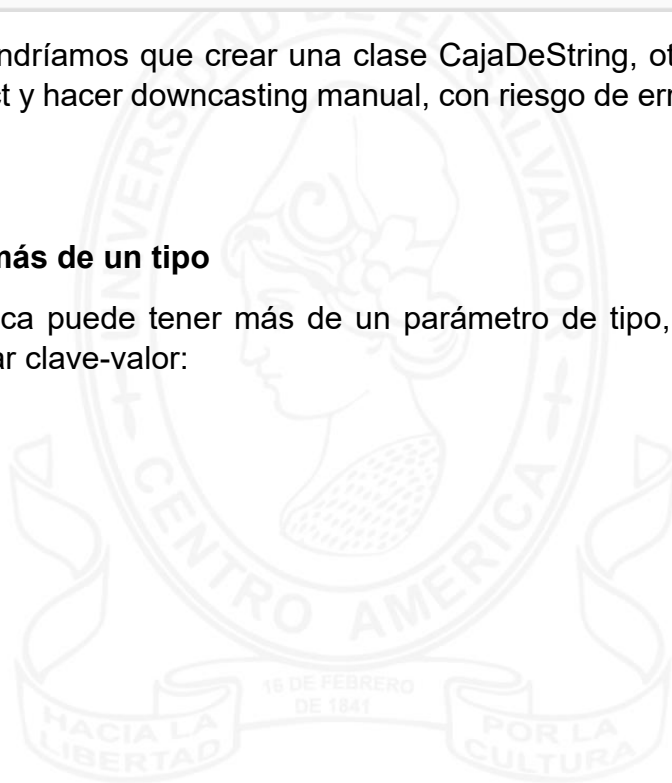
```
Caja<String> cajaTexto = new Caja<>();  
cajaTexto.guardar("Hola mundo");  
System.out.println(cajaTexto.obtener());
```

```
Caja<Integer> cajaNumero = new Caja<>();  
cajaNumero.guardar(100);
```

Sin genéricos, tendríamos que crear una clase CajaDeString, otra CajaDeInteger, etc., o usar Object y hacer downcasting manual, con riesgo de errores en tiempo de ejecución.

Genéricos con más de un tipo

Una clase genérica puede tener más de un parámetro de tipo, por ejemplo para representar un par clave-valor:



java

```
public class Par<K, V> {  
  
    private K clave;  
    private V valor;  
  
    public Par(K clave, V valor) {  
        this.clave = clave;  
        this.valor = valor;  
    }  
  
    public K getClave() { return clave; }  
    public V getValor() { return valor; }  
}
```

Métodos genéricos

También es posible declarar un método genérico dentro de una clase normal, sin que toda la clase sea genérica:

java

```
public static <T> void imprimirElemento(T elemento) {  
    System.out.println("Elemento: " + elemento);  
}
```

Características principales

- Se declaran con un parámetro de tipo entre signos < > (por ejemplo, <T>).
- El compilador valida los tipos usados, evitando errores de ClassCastException en tiempo de ejecución.



- Permiten reutilizar una misma clase o método con diferentes tipos de datos.
- Son la base de las colecciones de Java, como `List<T>`, `Map<K, V>` y `Set<T>`.

Ventajas de los genéricos

1. **Seguridad de tipos:** los errores de tipo se detectan al compilar, no al ejecutar el programa.
2. **Eliminación de downcasting:** no es necesario hacer casting manual al recuperar un valor.
3. **Reutilización de código:** una sola clase o método sirve para múltiples tipos de datos.
4. **Legibilidad:** el código expresa claramente qué tipo de datos maneja cada estructura.

Resumen para recordar

- Genéricos = clases y métodos parametrizados por tipo.
- Se escriben con `<T>`, `<K, V>`, etc., y Java verifica los tipos en tiempo de compilación.
- Evitan el uso de `Object` y el downcasting manual.
- Son la base de las colecciones de la API estándar de Java.

